

Functional Test Report

UART-Controlled Low-Power Multi-Clock Digital System

This report documents functional simulation test cases for the four supported system commands: Register File Write, Register File Read, ALU operation with operands, and ALU operation without new operands.

Prepared by: Mohamed Elsayed Mohamed Abdelmoneim

Digital IC Design Project

Contents

1. Project Overview	1
2. Supported Command Protocol	1
3. Test Strategy	1
4. Functional Test Cases	1
Test Case 1 - Register File Write Command	2
Test Case 2 - Register File Read Command	3
Test Case 3 - ALU Command with Operands	4
Test Case 4 - ALU Command without New Operands	5
Additional Configuration Check - UART Configuration Register	6
5. Conclusion	7

List of Figures

1	Register File Write command: AA → 09 → 3A.	2
2	Register File Read command: BB → 09, returning 0x3A.	3
3	ALU command with operands: CC → 4B → 3A → 02.	4
4	ALU command without new operands: DD → 04.	5
5	Reading the UART configuration register REG2 at address 0x02.	6

1. Project Overview

The project implements a UART-controlled digital system that receives commands from an external master, processes them internally, and returns the requested result through the UART transmitter. According to the received command, the controller can access the internal Register File or execute an ALU operation.

The design operates across two main clock domains. The **REF_CLK** domain contains the Register File, ALU, clock-gating block, and **SYS_CTRL**. The UART side contains the UART transmitter/receiver, clock dividers, and pulse-generation logic. Data transferred between the two clock domains is handled through synchronizers and an asynchronous FIFO.

Main data flow: UART RX → Data Synchronizer → SYS_CTRL → RegFile / ALU → Async FIFO → UART TX.

The design also uses clock gating on the ALU clock. The ALU clock is enabled only when an ALU operation is required, reducing unnecessary switching activity while the ALU is idle.

2. Supported Command Protocol

Command	Code	Frames	Function
Register File Write	0xAA	3	Write one data byte into a selected Register File address.
Register File Read	0xBB	2	Read one Register File address and return the stored byte through UART TX.
ALU with operands	0xCC	4	Receive Operand A, Operand B, and an ALU function code, then execute the operation.
ALU without operands	0xDD	2	Execute a new ALU function using the operands already stored in the operand registers.

The normal Register File address range used by external read/write commands is 0x04 to 0x0F. Addresses 0x00 to 0x03 are reserved for ALU operands and system configuration registers.

3. Test Strategy

The testbench sends complete UART frames and checks the response at system level. The most important signals are the decoded RX data, **SYS_CTRL** state transitions, Register File control signals, ALU controls, result-valid indications, and UART transmit activity.

Test	Command	Applied sequence
1	Register File Write	AA → 09 → 3A
2	Register File Read	BB → 09
3	ALU with operands	CC → 4B → 3A → 02
4	ALU without operands	DD → 04

Test Case 1 - Register File Write Command

Command code: 0xAA

Objective: Verify that the controller recognizes 0xAA, receives an address and a data byte, and performs a valid Register File write operation.

Applied sequence: 0xAA → 0x09 → 0x3A

The waveform shows the controller moving from IDLE to WRITE ADDR and then WRITE DATA. The selected address becomes 0x09, the write data becomes 0x3A, and WrEn is asserted to complete the transaction.

Result: PASS. The value 0x3A is written to Register File address 0x09.

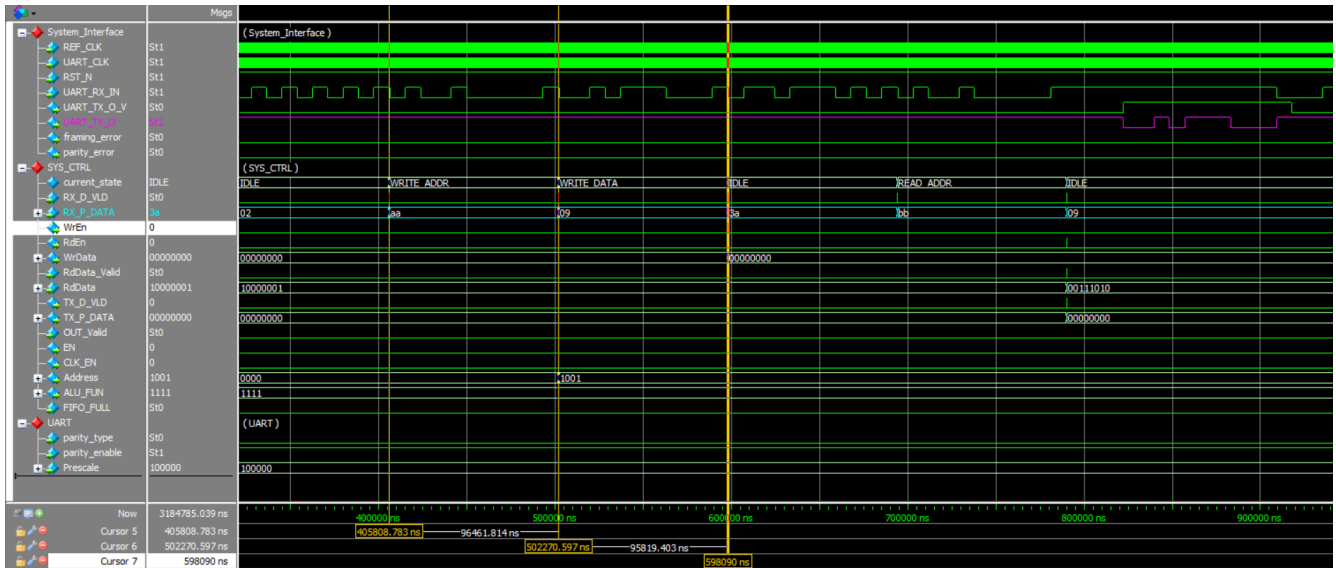


Figure 1: Register File Write command: AA → 09 → 3A.

Test Case 2 - Register File Read Command

Command code: 0xBB

Objective: Verify that the controller recognizes 0xBB, reads the selected Register File address, and transfers the stored value to the UART transmit path.

Applied sequence: 0xBB → 0x09

The controller enters the READ ADDR state and asserts RdEn. The observed RdData is 0x3A, confirming that the value written in Test Case 1 is stored correctly. UART TX activity follows to return the byte to the master.

Result: PASS. Reading address 0x09 returns 0x3A, matching the previous write operation.

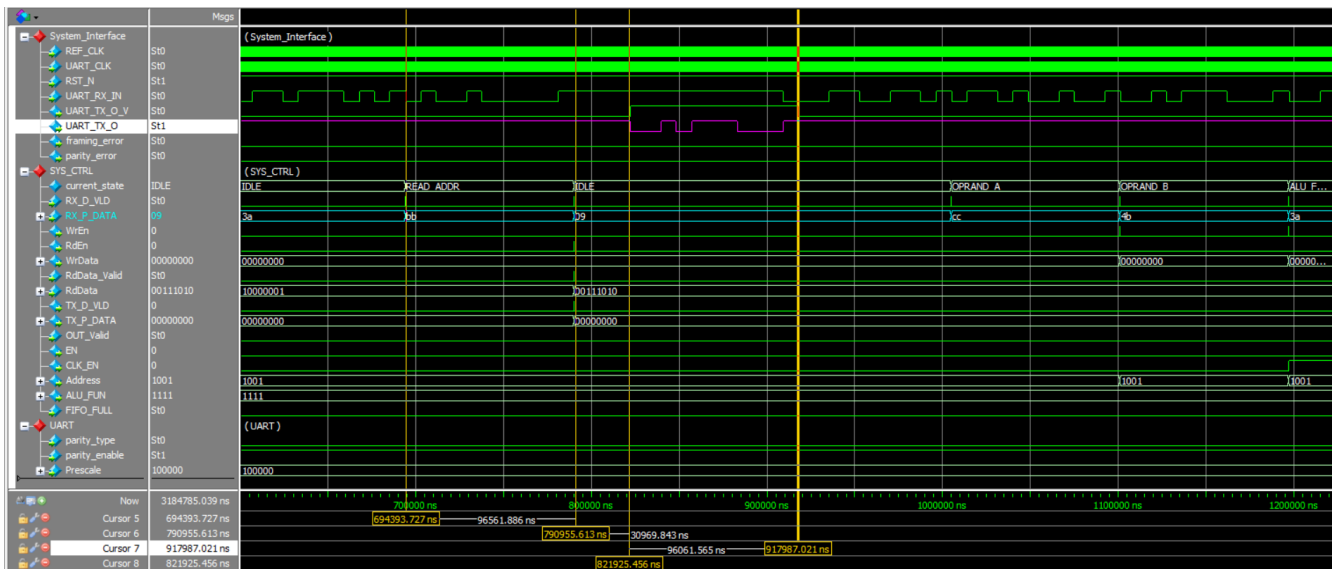


Figure 2: Register File Read command: BB → 09, returning 0x3A.

Test Case 3 - ALU Command with Operands

Command code: 0xCC

Objective: Verify the four-frame ALU command in which new Operand A, Operand B, and the ALU function are received before execution.

Applied sequence: 0xCC → 0x4B → 0x3A → 0x02

After decoding 0xCC, SYS_CTRL passes through the operand-A, operand-B, and ALU-function states. The received operands are stored, function code 0x02 is accepted, and the ALU clock/enable controls are activated before the result is transferred toward UART TX.

Result: PASS. All four frames are received in the expected order and the ALU execution sequence starts correctly.

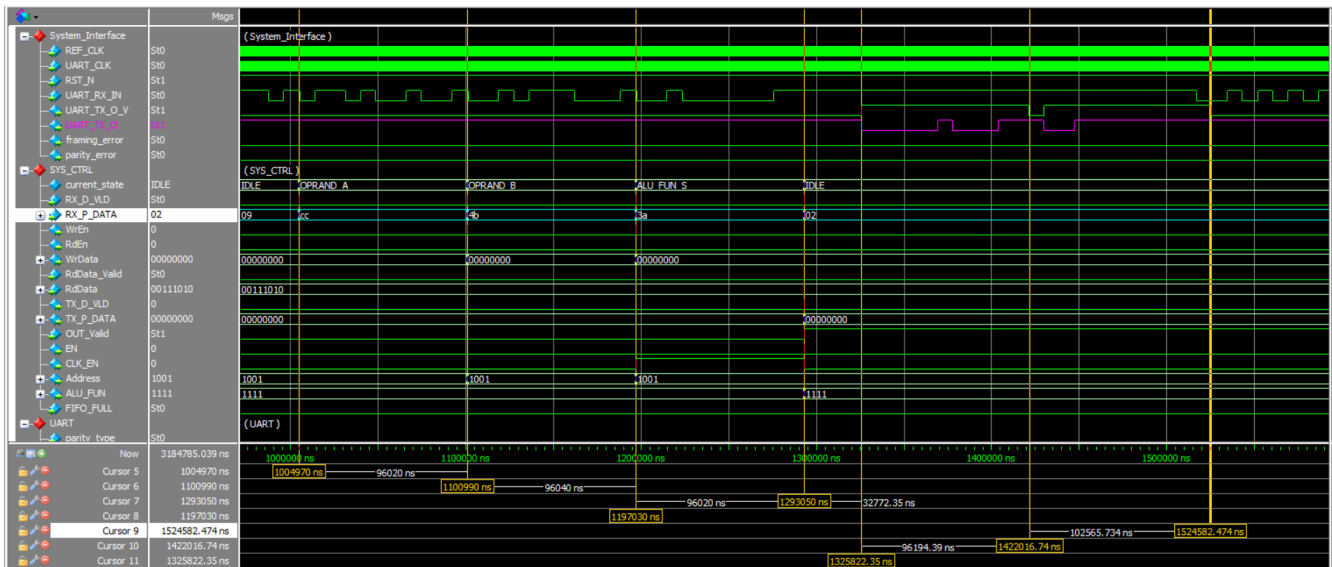


Figure 3: ALU command with operands: CC → 4B → 3A → 02.

Test Case 4 - ALU Command without New Operands

Command code: 0xDD

Objective: Verify that the controller can execute an ALU function using operands already stored in the Register File, without receiving new operand bytes.

Applied sequence: 0xDD → 0x04

Only the command byte and ALU function byte are required. After receiving 0xDD, the controller goes directly to the ALU-function state instead of requesting Operand A and Operand B again. The previously stored operands are reused and the result is sent through the UART transmit path.

Result: PASS. The controller skips operand loading and starts the ALU operation using the stored operands.

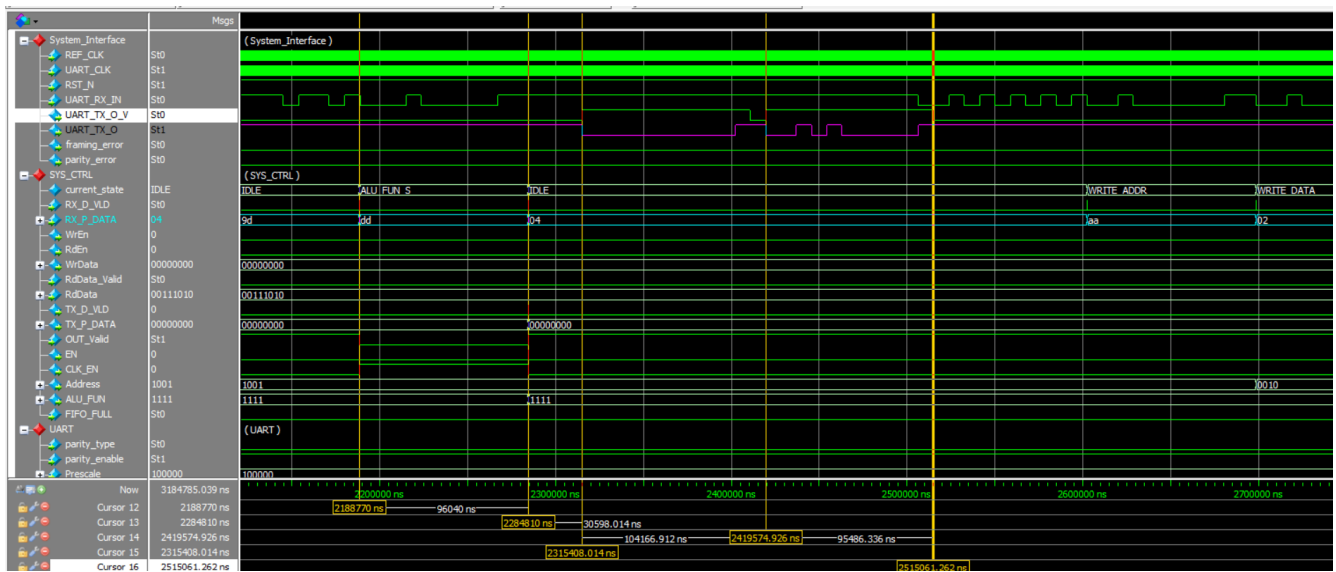


Figure 4: ALU command without new operands: DD → 04.

Additional Configuration Check - UART Configuration Register

Reserved Register File address: REG2 = 0x02

Purpose: Verify access to the reserved configuration area of the Register File before normal command execution.

Checked address: REG2 at 0x02

REG2 stores UART configuration fields, including parity control and the receive prescale value. The waveform shows a read from address 0x02 and the returned value 0x81, confirming that the configuration register can be accessed correctly during simulation.

Observed value: 0x81. The reserved UART configuration register is readable through the normal system datapath.

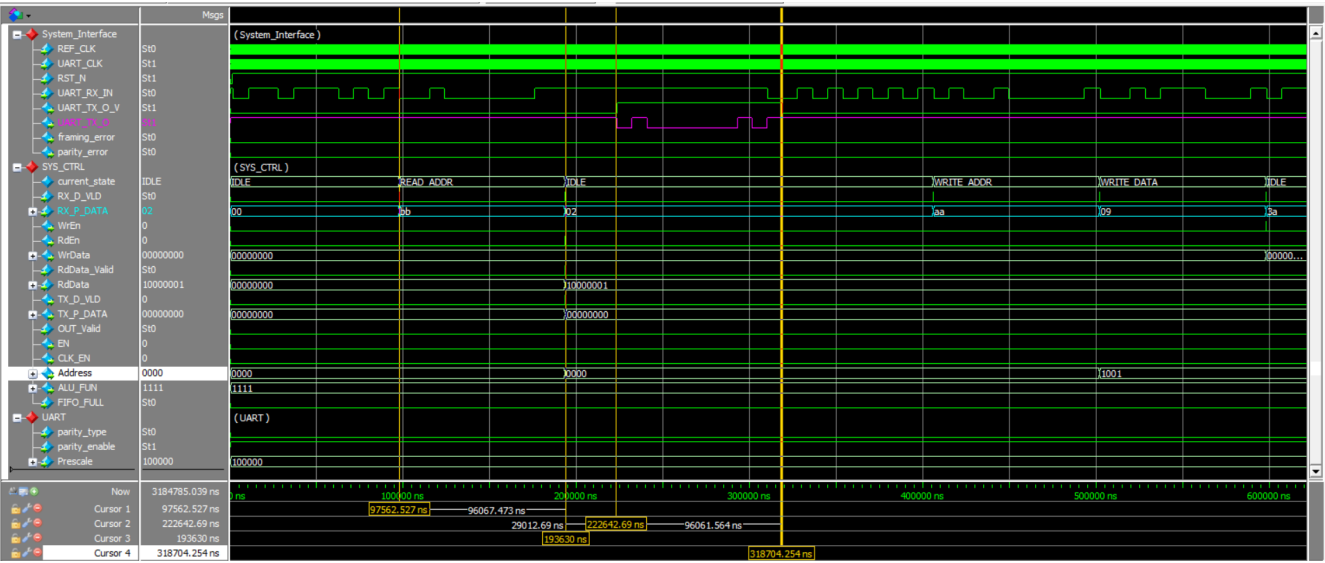


Figure 5: Reading the UART configuration register REG2 at address 0x02.

5. Conclusion

The simulation covers all four required command types. The Register File write/read path is verified using address 0x09, while both ALU command formats are recognized with the expected controller state sequences. The waveforms also demonstrate the interaction between UART reception, system control, Register File/ALU operation, clock-domain crossing, and UART transmission.

Overall status: PASS. The required four command scenarios are covered, and an additional read of the UART configuration register is included as a supplementary system check.